PAC-MAN

Basics of Programming 3.

Project Specification

Balázs Tari

E3K6W5

October 16, 2026

Revised: November 6, 2026

Contents

1 Task Description

Pac-Man is a two-dimensional desktop arcade game. The player controls the yellow figure in a labyrinth, whose task is to collect all dots while avoiding the ghosts moving around the map. If Pac-Man collects one of the 4 'Power Pellets' located in the labyrinth, the player gains the ability to eat ghosts for 6 seconds, earning additional points.

The software's graphical user interface is built using the Java Swing framework, the persistent storage of the game state is managed via file handling, and the code quality is ensured by automated JUnit unit tests.

2 Use-Cases

1. **Starting and controlling the game:** The user can start a new game from the menu and control Pac-Man within the labyrinth using the WASD keyboard keys.
2. **Collecting dots and victory:** As Pac-Man passes through the dots, they disappear, and the score increases. If all dots are consumed, the player wins and proceeds to the next level. At higher levels, the difficulty of the game increases.
3. **Collision detection with ghosts:** If the character collides with a ghost, it loses a life. If the number of lives reaches zero, the game ends (Game Over). Upon starting a new game, the player always begins at the first level.
4. **Saving high-scores:** At the end of a game, the system records and saves the achieved score to a file.
5. **Saving game state:** The game can be saved at any time and state, allowing the player to load and resume it later from a file.
6. **Level editor mode:** In addition to the base game, the user has the ability to design their own custom maps, save them, and load previously created maps to play on.

3 Solution Overview

1. **Applied technological solutions:** Java SE, Swing (JFrame, JPanel, Graphics for low-level drawing, `javax.swing.Timer` driving the game loop at 60 ticks per second), and JUnit 5 for automated testing.
2. **File formats:** A simple CSV file for storing the high-score table, a plain-text tile picture (one character per tile) for map layouts, and Java binary serialization for saved game states.

4 Detailed Architecture and Class Structure

Core Game Model

- **GameEngine:** The central controller of the game. It runs the game loop (see Section **??**), updates the game state (positions, collisions), and manages the score, remaining lives, and the `currentLevel`. It is also responsible for saving and loading the active game state (utilizing `java.io.Serializable`). As the level increases, the duration of the ghosts' Frightened state progressively decreases to increase difficulty; the normal movement speed of Pac-Man and the ghosts stays the same on every level. A new game is started from a `LevelData` object built by the `LevelManager`, always with score 0, 3 lives, and level 1. The engine owns one shared `java.util.Random` for the ghosts' random choices; a second constructor accepts a given `Random`, so tests can pass a fixed seed and get the same choices on every run.

- **Labyrinth:** The class representing the map. It stores the type of every tile (wall, dot, power pellet, empty) in a two-dimensional array, indexed as `grid[row][column]`. A tile is walkable if it is not a wall, and any position outside the array counts as a wall, so nothing can leave the map. An intersection is a walkable tile with at least three walkable neighbours. The `Labyrinth` does not read files; levels are loaded by the `LevelManager`.

- **GameObject** (Abstract Base Class): Encapsulates common properties for all moving entities (X and Y coordinates, speed, direction). Positions are **tile coordinates**: `x` is the column and `y` is the row of the maze, and `speed` is the number of whole tiles one step covers. Besides the current tile, every object remembers two more:

  - its **start tile** (`startX`, `startY`, `final`), where `resetToStart()` puts it back after Pac-Man loses a life, standing still;
  - its **previous tile** (`prevX`, `prevY`), saved by `rememberPosition()` at the start of every step, even if the object does not move. The view uses it to slide the figure smoothly between two tiles, and the collision check uses it to detect two figures passing through each other.

  - **PacMan** (Child Class): Represents the state of the player's character and handles its input logic: the latest key press is buffered as a `Command` and run at the next step. It overrides `resetToStart()`: after calling the parent version it also drops the buffered command, so a key pressed before dying does not start Pac-Man moving again right away.
  - **Ghost** (Child Class): Represents the state of the ghosts and manages their movement. `possibleDirections()` lists the walkable directions from the current tile, without turning back, unless the ghost is in a dead end. `chooseDirection()` picks one of them; for now the pick is random, later the target tile computed by the ghost's strategy object decides (see Section **??**). In a corridor or a corner only one direction is left, so a real choice only happens at intersections. State changes are triggered by events, for example `frighten()` when Pac-Man eats a power pellet.

- **LevelData:** A plain data holder for the content of one map file: the `Labyrinth`, Pac-Man on his start tile, and the ghosts on their start tiles. The `LevelManager` creates it, and the `GameEngine` is built from its parts.

UI and Management Classes

- **MainFrame (JFrame):** The application's main window utilizing a CardLayout to switch between the menu, the active game, and the editor.

- **MainMenu (JPanel):** The start screen providing options to start a new game, load a saved game, view high scores, or open the map editor.

- **GamePanel (JPanel):** The active gameplay screen that bridges the Model and View. It turns WASD key presses into `MoveCommand` objects, receives the game state from the `GameEngine`, and renders the maze and entities in its `paintComponent` method. Figures are drawn between their previous and current tile according to the engine's step progress, so movement looks smooth although the model works in whole tiles. Frightened ghosts are drawn white, the others red. The heads-up line shows the score, the remaining lives, and `GAME OVER` when no life is left.

- **MapEditor (JPanel):** A graphical interface with a clickable grid allowing the user to draw custom mazes (placing walls, dots, power pellets, Pac-Man's start tile, and the start tiles of the four ghosts).

- **LevelManager (Factory):** A dedicated, stateless class that reads a map file into a `LevelData` (maze, Pac-Man, and ghosts) and writes a `LevelData` back to a map file in the same format (see Section **??**). It also manages the high-score table.

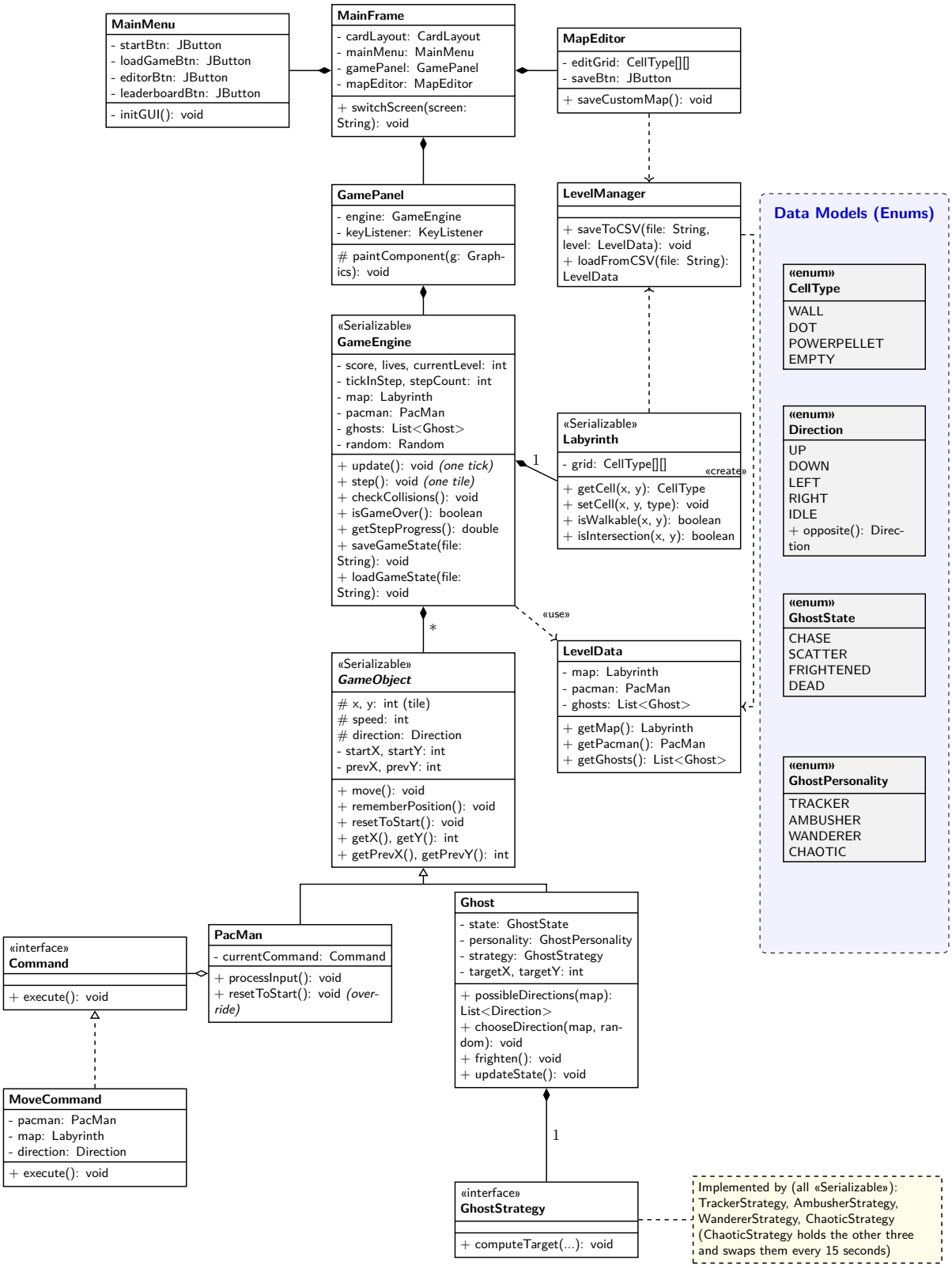


Figure 1: UML Class Diagram of the Complete Game Architecture

4.1 Game Loop, Movement and Collisions

The model works in whole tiles, but the screen is redrawn 60 times per second. The two are kept apart by separating a *tick* from a *step*:

- **Tick:** The Swing `Timer` in the `GamePanel` fires every 1000/`TICKS_PER_SECOND` ms (≈16 ms), calls `GameEngine.update()`, then `repaint()`. One `update()` is one tick.
- **Step:** Every `TICKS_PER_STEP` (= 10) ticks, `update()` calls `step()`, which moves every figure by at most one tile. With 60 ticks per second this is 6 tiles per second. All game rules (input, movement, collisions, scoring) run in `step()`, so the unit tests call `step()` directly and do not depend on timing.
- **Smooth drawing (interpolation):** `getStepProgress()` returns how far the current step is, between 0.0 and 1.0. The `GamePanel` draws each figure at $(previous + (current - previous) \cdot progress) \cdot TILE$ pixels. The pixel calculation lives only in the view; the model never deals with pixels. As a side effect the picture runs up to one step (≈166 ms) behind the model.
- **Durations:** Every duration (Frightened time, the Chaotic ghost's 15-second switch) is converted from seconds with `TICKS_PER_SECOND`, so changing the tick rate needs only one constant.

Order of one step:

1. If the game is over, nothing happens.
2. Pac-Man runs his buffered command, which turns him if the tile in the wished direction is open, then steps one tile unless a wall is ahead.
3. Each ghost chooses a direction and steps. A frightened ghost moves only on every `FRIGHTENED_STEP_INTERVAL` (= 2) steps, so it is half as fast; on the skipped steps it still calls `rememberPosition()`.
4. Collisions are checked once everyone has moved.

Before moving, every figure saves its previous tile with `rememberPosition()`, even if it stands still. Otherwise the view would replay the last slide and the collision check would compare old tiles.

Ghost movement rules: A ghost never turns back by 180 degrees, except in a dead end, where turning back is the only way out, and once when it becomes frightened. In a corridor or a corner only one direction is left, so a ghost really decides only at intersections. Until the personality strategies are implemented, the choice is random.

Collisions:

- **Dots and power pellets:** If Pac-Man ends a step on a `DOT` (10 points) or a `POWERPELLET` (50 points), the score increases and the tile becomes `EMPTY`. A power pellet also calls `frighten()` on every ghost.
- **Pac-Man and a ghost** meet if, after the step, they stand on the same tile, *or* they swapped tiles: Pac-Man stands where the ghost stood before *and* the ghost stands where Pac-Man stood before. Both halves of the swap condition are required; one alone would also be true when Pac-Man simply follows a ghost. This check is exact because every figure moves at most one tile per step.
- **Losing a life:** On a deadly collision, `lives` decreases by one and Pac-Man and *all* ghosts return to their start tiles with `resetToStart()`; eaten dots stay eaten. At most one life can be lost per step: the collision loop stops after the first hit.

- **Game Over:** `isGameOver()` is computed from the lives (`lives <= 0`) instead of being stored in a separate flag, so the two can never contradict each other. When it is true, `step()` returns immediately and nothing moves any more.

5 Design Patterns

To ensure the codebase meets university standards and demonstrates solid object-oriented principles, the implementation must explicitly utilize the following design patterns:

- **Ghost Movement (Strategy Pattern):** The target-tile calculation of each personality lives in its own class (`TrackerStrategy`, `AmbusherStrategy`, `WandererStrategy`, `ChaoticStrategy`), all implementing a common `GhostStrategy` interface. The `GhostPersonality` enum decides which strategy a ghost receives, and the `Ghost` holds it as a field (composition instead of inheritance). This matters for the Chaotic ghost: an object cannot change its class at runtime, but a ghost can swap the strategy object it holds. The `ChaoticStrategy` keeps its own state (the 15-second timer and the currently active strategy) and switches between the other three. Since every ghost is saved together with its strategy, the strategy classes must implement `java.io.Serializable`.

- **Ghost Behavior (Finite State Machine):** The ghosts' current mode is managed using a `GhostState` enum (`CHASE`, `SCATTER`, `FRIGHTENED`, `DEAD`). The transitions between these states are strictly governed by the game events shown in Figure **??**. Each event has its own method on `Ghost` (for example `frighten()` for "Power Pellet"), instead of one public setter that accepts any state. The rule of which state may follow which then lives inside the ghost: `frighten()`, for instance, leaves a `DEAD` ghost alone. The parameterless `updateState()` handles the timed `CHASE`/`SCATTER` switches.

  Ghost State Definitions:

  - **CHASE:** Each ghost uses its unique artificial intelligence to actively target and corner Pac-Man.
  - **SCATTER:** Ghosts temporarily stop pursuing Pac-Man and retreat to patrol their own designated corners of the maze.
  - **FRIGHTENED:** After Pac-Man consumes a Power Pellet, the ghosts turn white, turn around once, move randomly at half speed, and become vulnerable to being eaten. A ghost that is already frightened does not turn around again on a new pellet, and a `DEAD` ghost is not affected.
  - **DEAD (Eaten):** Reduced to just a pair of floating eyes, the ghost rushes back to the central Ghost House to respawn and regain its body.

- **Player Input (Command Pattern & Input Buffering):** Movement actions are encapsulated into `MoveCommand` objects implementing a common `Command` interface with an `execute()` method. Instead of one class per direction, a single `MoveCommand` stores the wished `Direction`; the `GamePanel` creates one for each WASD key press and buffers it inside the `PacMan` instance (`currentCommand`). The command is executed at every step: if the tile in the wished direction is open, Pac-Man turns; if it is a wall, his current direction is kept and the wish is tried again at the next step. This explicitly solves the *corner pre-turning* (input buffering) mechanic: a turn pressed a little early happens at the first tile where it is possible.

- **Model-View-Controller Architecture:** The logical game state (Model) must be strictly separated from the Swing rendering (View). Logical entities like `PacMan` or `Ghost` must not contain any UI code, `javax.swing` imports, or `java.awt.Graphics` references.

- **Factory Pattern:** The creation of a level from a map file is handled by the dedicated `LevelManager` class. It parses the file, builds the `Labyrinth`, creates Pac-Man and the ghosts on their start tiles, and returns them together as a `LevelData`. This prevents the

`GameEngine` from being bloated with initialization logic, and the engine never needs to know the file format.

- **Observer Pattern:** The UI rendering component (`JPanel`) should observe the `GameEngine`. When the game state updates on a timer tick, the engine notifies the view to trigger a `repaint()`, decoupling the game loop from the graphics pipeline.

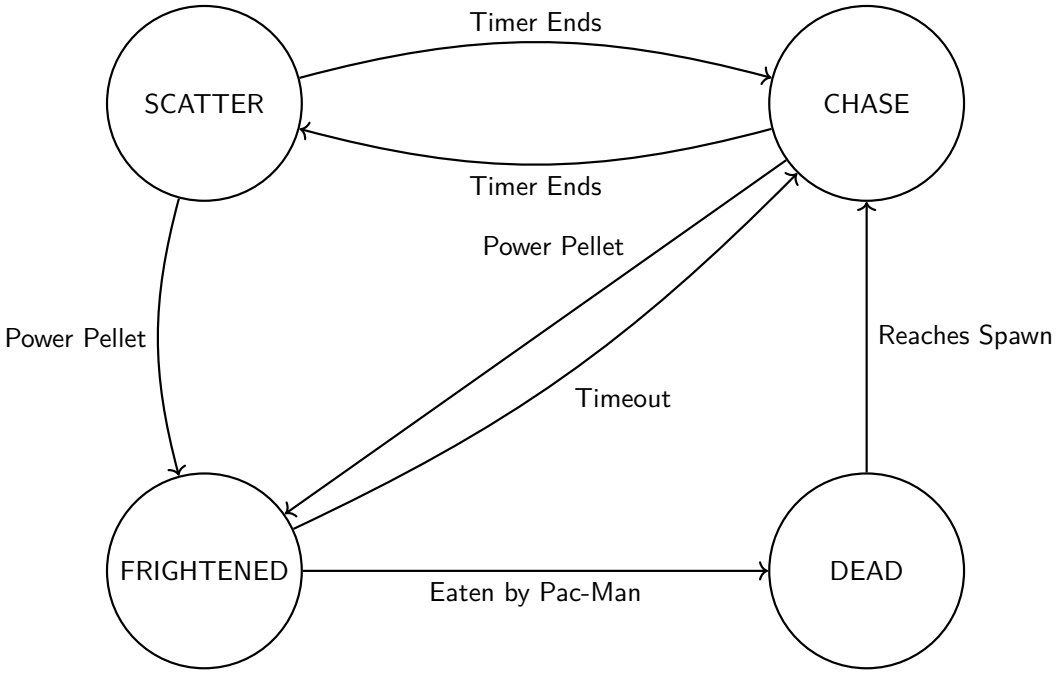


Figure 2: Finite State Machine (FSM) of Ghost Behaviors

6 Unit Testing

The project must include a minimum of **10 automated JUnit 5 tests**. These tests must strictly focus on the core logical components (the Model layer, such as `GameEngine` and `Labyrinth`) without instantiating the Swing graphical user interface.

Every test case must follow the industry-standard **Arrange-Act-Assert (Given-When-Then)** methodology:

- **Arrange (Given):** Initialize the `GameEngine`, load a controlled, minimalist test map matrix, and position the entities manually.
- **Act (When):** Trigger a specific game mechanic (e.g., execute a movement command, run one `step()`, or force a collision). Tests call `step()` rather than `update()`, because a test checks a game rule, not the timing; and they create the engine with a fixed-seed `Random`, so the ghosts' "random" choices are the same on every run.
- **Assert (Then):** Verify the expected changes in the game state using JUnit `Assertions` (e.g., checking if the score increased, coordinates changed, or ghost states updated).

Mandatory Test Scenarios

The test suite must cover critical gameplay mechanics, including but not limited to:

- **Wall Collision:** *Given* Pac-Man is adjacent to and facing a `WALL` tile, *When* a move command is processed, *Then* Pac-Man's (X, Y) coordinates must remain unchanged.
- **Dot Consumption:** *Given* Pac-Man moves onto a `DOT` tile, *When* the engine updates the state, *Then* the score must increase, and the tile must change to `EMPTY`.
- **Ghost Frightened Transition:** *Given* Pac-Man consumes a `POWERPELLET`, *When* the collision is resolved, *Then* all ghosts must transition from `CHASE`/`SCATTER` to the `FRIGHTENED` state.
- **Death Mechanic:** *Given* Pac-Man and a `CHASE` ghost stand on the same tile (or swap tiles during one step), *When* collisions are evaluated, *Then* the player's life counter must decrease by 1, and the positions must reset to their respective spawn points.
- **Game Over:** *Given* Pac-Man has one life left, *When* he collides with a `CHASE` ghost, *Then* `isGameOver()` must be true, and further steps must not move any figure.

7 I/O and Persistence Contract

The application is following these persistance protocols in order to ensure robust data handling:

1. High-Score Table

- **File:** `highscores.csv`
- **Responsibility:** Handled by the `LevelManager` class, which serves as the general I/O utility for the application.
- **Data Structure:** Each line must strictly follow the format: `PlayerName;Score;Date`.
- **Player Identity:** The system must not interrupt the player with a text input prompt to ask for their name. Instead, it must automatically fetch the identity of the user currently logged into the operating system using `System.getProperty("user.name")`.

2. Active Game State

Saving the complex state of an ongoing game (the exact position of Pac-Man, the current `GhostState` of every ghost, the remaining Frightened timer, the active level, and the state of every individual dot on the map) into a plain text file would require highly error-prone parsing.

Therefore, the active game state is saved using **Java Binary Serialization**:

- **File:** `savestate.dat`
- **Implementation:** All Core Model classes (`GameEngine`, `Labyrinth`, `GameObject`, `PacMan`, `Ghost`, the `GhostStrategy` implementations, and data Enums) must implement the `java.io.Serializable` interface. `LevelData` is only passed from the `LevelManager` to the `GameEngine` and is not saved, so it does not need to. The engine's `java.util.Random` is itself serializable, and the plain counters (`tickInStep`, `stepCount`) and every figure's start and previous tile are saved automatically.
- **Execution:** The `GameEngine.saveGameState()` method writes the entire engine instance to a binary file using an `ObjectOutputStream`. Loading a game simply reads the object back into memory using an `ObjectInputStream`, perfectly restoring the exact state without manual data mapping.

3. Maps

- **Format:** Both the built-in levels and the maps made in the `MapEditor` are plain-text files that look like the maze itself: one maze row per line, one character per tile. The `LevelManager` reads them line by line with `loadFromCSV` and writes them with `saveToCSV`. The built-in first level is `maps/level1.txt`; the path is relative, so the game must be started from the project folder.
- **Symbols:**

Character	Meaning	Stored as
`#`	Wall	`CellType.WALL`
`.`	Dot	`CellType.DOT`
`o`	Power pellet	`CellType.POWERPELLET`
`_`	Empty floor	`CellType.EMPTY`
`P`	Pac-Man's start tile (exactly one)	`PacMan` on an `EMPTY` tile
`T`	Start tile of a `TRACKER` ghost	`Ghost` on an `EMPTY` tile
`A`	Start tile of an `AMBUSHER` ghost	`Ghost` on an `EMPTY` tile
`W`	Start tile of a `WANDERER` ghost	`Ghost` on an `EMPTY` tile
`C`	Start tile of a `CHAOTIC` ghost	`Ghost` on an `EMPTY` tile

- **Why characters instead of `CellType` names:** A `CellType` matrix can only describe tiles, so it has no way to record where Pac-Man and the ghosts start. The character picture stores both, stays readable, and can be edited by hand. The ghosts' start tiles also mark the Ghost House, where `DEAD` ghosts return.

- **Validation:** Every row must have the same length, exactly one `P` is required, and any other character (including a space) is rejected with its line and column. Empty lines and spaces at the end of a line are ignored, which is why empty floor is written as `_` and not as a space.

- **Example** (a small maze with Pac-Man, a power pellet, and two ghosts):

```
#########
#P..o...#
#.##_##.#
#..T_A..#
#########
```